

PRACTICA 3: ALGORITMOS VORACES

Subgrupo 2 del C2:

Hugo Aguiar García 20%

Jose Daniel Galán Gómez 20%

Alfonso Maldonado González 20%

Adrián Rosado Rueda 20%

Felipe Ocaña Valenzuela 20%

ÍNDICE

PROBLEMA 1.1: RATONES Y AGUJEROS.....	2
Primera Solución (Greedy).....	2
Explicación del Algoritmo.....	2
Optimalidad y Eficiencia.....	2
Gráficas de Eficiencia.....	3
Segunda Solución (No Greedy).....	4
Explicación del Algoritmo.....	4
Optimalidad y Eficiencia.....	4
Gráficas de Eficiencia.....	5
Análisis de ambos algoritmos.....	6
Gráfica Comparativa de los 2 algoritmos del Problema Ratones y Agujeros.....	6
Análisis en base a la Gráfica y Datos de ambos problemas.....	6
PROBLEMA 2: Cámaras de vigilancia.....	7
Primera Y Única Solución (Greedy).....	7

PROBLEMA 1.1: RATONES Y AGUJEROS

Primera Solución (Greedy)

Explicación del Algoritmo

El **algoritmo** recibe **dos vectores** que contienen las **posiciones de los ratones** y de los **agujeros**. El primer paso consiste en **ordenar ambos vectores** de menor a mayor utilizando la función **sort** de la biblioteca estándar. Esta ordenación previa constituye el **núcleo de la estrategia voraz**, ya que **asegura que el i-ésimo ratón más a la izquierda se asigne al i-ésimo agujero más a la izquierda**, garantizando que las trayectorias no se crucen y se **minimice la distancia máxima**.

Una vez ordenados los datos, se **inicializa a 0 la variable *resultado.tiempo***, que se encargará de registrar el tiempo que tarda el ratón más lento. A continuación, se **recorren ambos vectores** simultáneamente mediante un **único bucle** utilizando el índice **i**, **emparejando** secuencialmente cada ratón con su agujero correspondiente.

Para cada iteración, se calcula el tiempo que invierte ese ratón calculando el valor absoluto de la diferencia entre sus posiciones (**$\text{abs}(\text{ratones}[i] - \text{agujeros}[i])$**). Si este tiempo calculado es estrictamente mayor que el tiempo máximo almacenado en ***resultado.tiempo***, se actualiza dicha variable. Simultáneamente, se introducen ambas posiciones de forma consecutiva en el vector ***resultado.ratonAgujero*** mediante la función ***push_back***. Al concluir el bucle, el algoritmo devuelve la estructura resultante con la asignación completa y el tiempo óptimo en el que todos los ratones quedan resguardados.

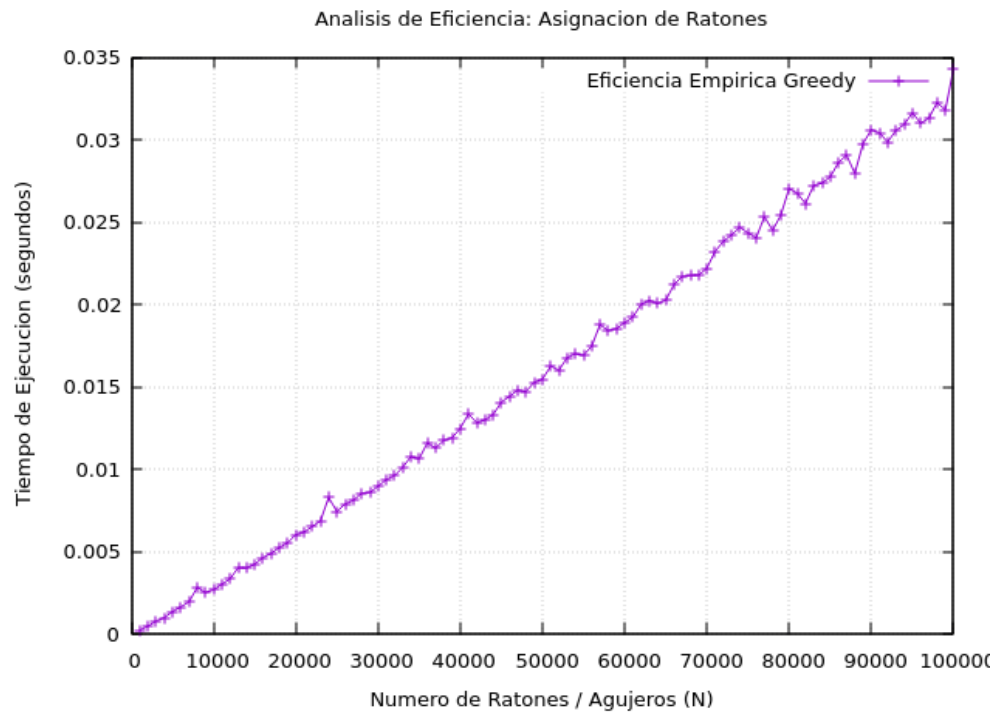
Optimalidad y Eficiencia

Para explicar esta optimización lo haremos por **reducción al absurdo**. Para ello diremos que **nuestro algoritmo (A) no es el mejor algoritmo** y por tanto **habrá otro algoritmo mejor (A*)**. Como en nuestro algoritmo A **no hay cruces entre ratones** es decir, el ratón más a la izquierda va al agujero más a la izquierda y así sucesivamente, **en el algoritmo A* tiene que haber al menos un cruce**.

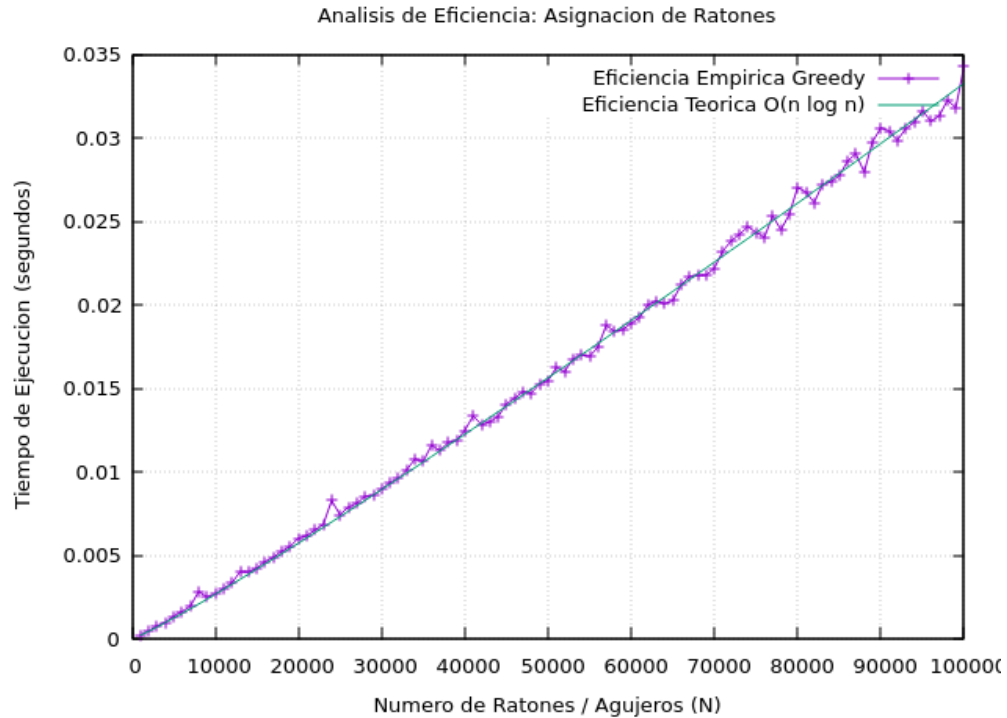
En un supuesto con solo **dos ratones** (de izquierda a derecha r_1 y r_2), y **dos agujeros** (a_1 y a_2), sí o sí se tendrían que cruzar. Al cruzarse el tiempo del que más tarde será **$\max(|m_2 - r_1|, |m_1 - r_2|)$** . Por simple geometría si tenemos dos rectas con dos puntos en cada una la distancia entre extremos siempre será mayor a las distancias paralelas. Por tanto el **algoritmo A*** no es el más óptimo entonces nuestro **algoritmo A** si que será el más óptimo.

El algoritmo greedy es **bastante eficiente**, su parte más costosa es la ordenación de los vectores con sort siendo su eficiencia **$O(n \log n)$** , para luego recorrer los dos vectores con un único bucle para emparejar cada ratón con su agujero correspondiente y calcular el tiempo máximo, siendo esto **$O(n)$** . La eficiencia total es **$O(n \log n) + O(n)$** pero tomamos únicamente en cuenta la parte con crecimiento más rápido, por tanto es **$O(n \log n)$** .

Gráficas de Eficiencia



Gráfica de Eficiencia Empírica del Algoritmo Greedy



Gráfica de Eficiencia Híbrida del Algoritmo Greedy

Segunda Solución (No Greedy)

Explicación del Algoritmo

El **algoritmo** recibe **dos vectores** que contienen las **posiciones de los ratones** y de los **agujeros**. En este caso, a diferencia del algoritmo voraz, no se toma directamente una **decisión óptima local**, sino que se prueban **todas las asignaciones** existentes entre **ratones** y **agujeros**.

Primero se inicializa la variable **resultado.tiempo** con el valor **INT_MAX**, de la librería **climits**, la cual va a permitir almacenar el mayor tamaño posible de un entero en **C++**, haciendo que cualquier solución real que se encuentre sea mejor que la inicial. Después se ordena el vector de agujeros para generar todas sus posibles permutaciones con la función **next_permutation**.

En cada **permutación**, el algoritmo recorre los ratones con un bucle y calcula el tiempo que tarda cada uno en llegar al agujero que le corresponde en esa asignación mediante **abs(ratones[i] - agujeros[i])**. De todos esos tiempos se guarda el mayor en la variable **tiempoMaximo**, ya que el objetivo es minimizar el tiempo que tarda el último ratón en resguardarse.

Si **tiempoMaximo** de la asignación actual es menor que el mejor tiempo guardado en **resultado.tiempo**, se actualiza dicho valor y se guarda esa asignación en **resultado.ratonAgujero**, introduciendo consecutivamente la posición del ratón y su agujero asignado. Al terminar todas las permutaciones, el algoritmo **devuelve** la **mejor asignación encontrada** y el **tiempo mínimo necesario** para que todos los ratones estén resguardados.

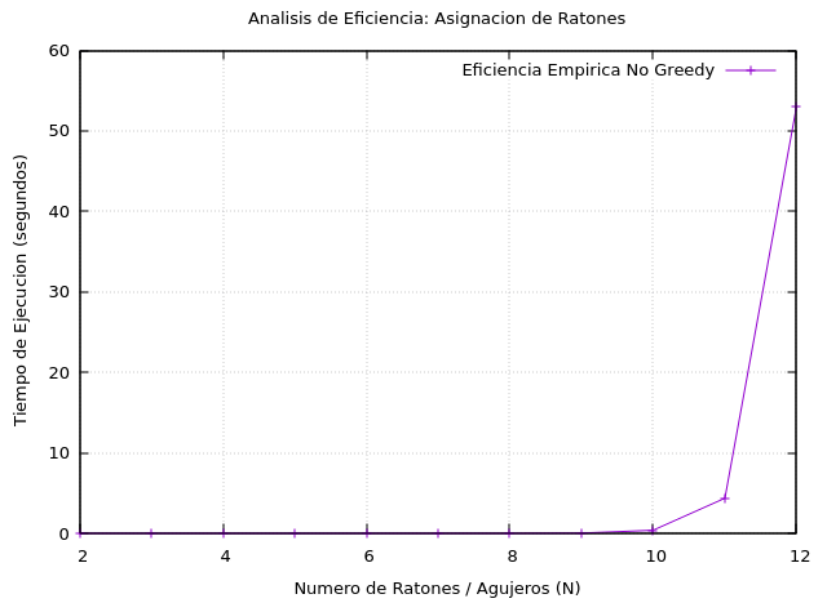
Optimalidad y Eficiencia

Este **algoritmo** es **óptimo**, ya que prueba **todas las asignaciones posibles** entre ratones y agujeros. Para cada una calcula el tiempo del ratón que más tarda y se queda con la asignación que tiene el menor tiempo máximo.

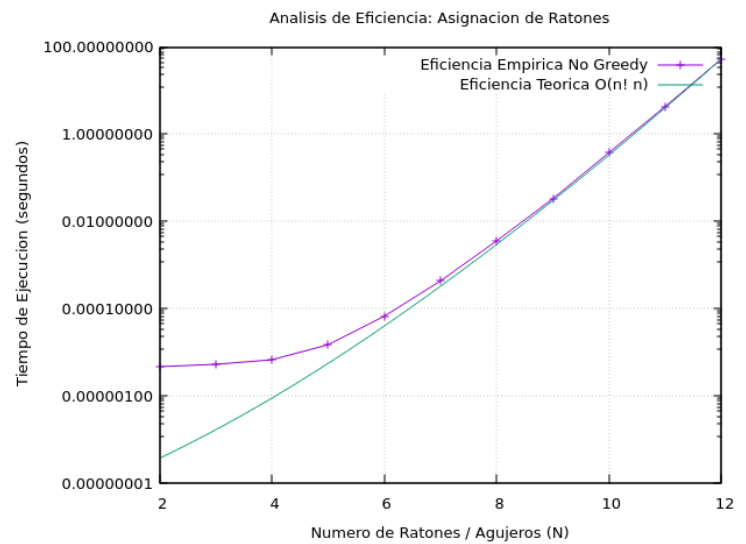
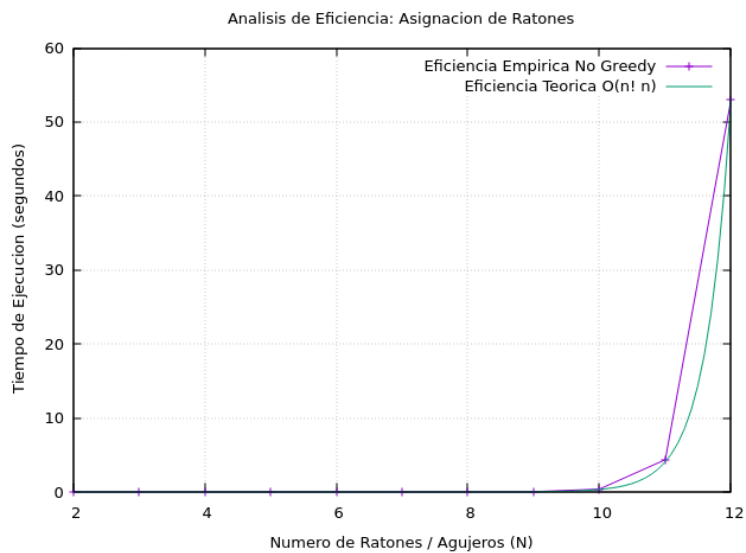
Por tanto, **no puede existir una solución mejor** que no haya sido considerada ya que el algoritmo realiza **todas las combinaciones posibles**, y si hubiese un caso con un tiempo menor el algoritmo también **la habría probado** y **la habría guardado** como mejor solución.

Pese a ser un **algoritmo óptimo**, no significa que sea muy **eficiente**, ya que en nuestro algoritmo consideramos todas las opciones existentes del problema, siendo que si hay **n ratones** y **n agujeros**, por tanto existen **n!** formas posibles de asignarlos, siendo que además que para cada asignación se recorren los **n** ratones para calcular el tiempo máximo, por tanto tenemos una eficiencia de **O(n!·n)**, siendo muchísimo menos eficiente que el algoritmo Greedy.

Gráficas de Eficiencia



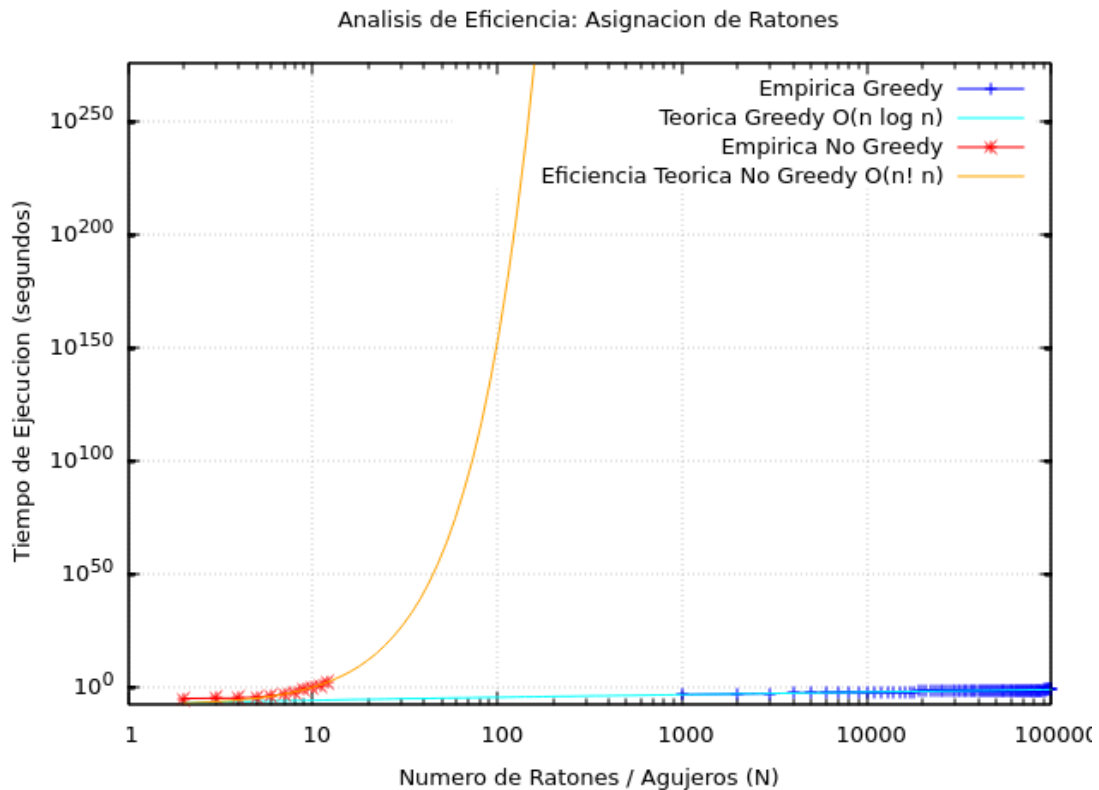
Gráfica de Eficiencia Empírica del Algoritmo No Greedy



Gráficas de Eficiencia Híbrida del Algoritmo No Greedy (escala lineal y logarítmica)

Análisis de ambos algoritmos

Gráfica Comparativa de los 2 algoritmos del Problema Ratones y Agujeros



Gráficas de Eficiencia Híbrida de ambos Algoritmos (en escala logarítmica)

Análisis en base a la Gráfica y Datos de ambos problemas

Las gráficas obtenidas permiten observar con claridad la enorme diferencia de rendimiento entre el algoritmo voraz y el algoritmo no voraz basado en fuerza bruta.

En el caso del algoritmo greedy, los tiempos de ejecución permanecen muy reducidos incluso para tamaños de entrada elevados. Esto se debe a que su coste dominante es la ordenación inicial de ambos vectores, con complejidad $O(n \log n)$, seguida de un recorrido lineal para realizar las asignaciones.

Por el contrario, el algoritmo exhaustivo presenta un crecimiento extremadamente rápido del tiempo de ejecución. Al considerar todas las posibles asignaciones entre ratones y agujeros, el número de combinaciones crece factorialmente, alcanzando complejidades del orden $O(n! \cdot n)$. Esto provoca que incluso para valores moderados de n los tiempos se disparen rápidamente, haciendo inviable su uso práctico.

Además, ambas soluciones obtienen el mismo valor óptimo, lo que demuestra que el algoritmo voraz no sólo es mucho más eficiente, sino también correcto y óptimo para este problema.

PROBLEMA 2: Cámaras de vigilancia

Primera Y Única Solución (Greedy)

Elementos de mi algoritmo voraz:

- Conjunto de Candidatos (C) : los vértices del grafo, ya que representan el sitio donde se sitúan las cámaras.
- Conjunto de Seleccionados (S): los vértices donde están situados las cámaras.
- Función Solución: todas las aristas están cubiertas, es decir, en los vértices hay cámaras puestas de tal manera que vigilan todos los pasillos, las aristas del grafo.
- Función de Factibilidad: siempre es posible encontrar una solución, pero no garantiza que esta sea óptima.
- Función Selección: el vértice con más vértices adyacentes.
- Función Objetivo: número de cámaras instaladas y sus posiciones.

Explicación del algoritmo:

1. El algoritmo tiene como parámetros las aristas, que es un vector de vectores de enteros (estos vectores son las parejas de vértices que forman las aristas) y los vértices seleccionados para ponerles las cámaras en ese momento que es un vector de enteros (este estará inicializado pero vacío).
2. Se crea la matriz de adyacencia del grafo (que sería el conjunto de vertices y aristas que me pasan) para saber qué vértices es de mayor grado.
3. Se mete ese vértice en el vector de seleccionados.
4. Se borran las aristas que se forman con ese vértice
5. Si aún quedan pasillos por cubrir, se vuelve a hacer el algoritmo pasándole como parámetros las aristas restantes y los vértices seleccionados.
6. Sino, se devuelve el vector de vértices seleccionados. El tamaño de este es el número de cámaras que se pone en el grafo.

Eficiencia teórica:

Línea 14-22: $O(n_a)$

Línea 27-31: $O(n_a)$

Línea 37-46: $O(n_v^2)$

Línea 55-62: $O(n_a)$

Línea 67-69: $O(n_v)$

Línea 70-71: $O(n_v)$

Línea 78-88: $O(n_v^2)$ (En el peor caso)

Orden del algoritmo= $O(3n_a + 2n_v + 2n_v^2)$

Eficiencia empírica:

Número de Vértices (nv)	Tiempo real (ms)
100	0.15
500	3.82
1000	15.24
2000	61.10
5000	384.45
10000	1528.20

Comparación de soluciones:

Tamaño (n)	Cámaras (Voraz)	Cámaras (FB - Óptimo) +4	Tiempo FB (Segundos) +4
4	1	1	$5 \cdot 10^{-6}$
8	2	4	$3.4 \cdot 10^{-5}$
12	4	7	0.000734
16	8	10	0.006366
20	10	13	0.109319

24	13	17	1.92321
28	17	18	37.4678

Análisis de la efectividad

El objetivo de este análisis es evaluar qué tan cerca se queda nuestra heurística voraz de la solución perfecta.

1. Calidad de la Solución

Como se observa en la tabla, nuestro algoritmo voraz basado en la selección del **vértice de grado máximo** es una aproximación muy sólida. Aunque no siempre alcanza el valor óptimo, la diferencia es pequeña en términos de recursos instalados:

- Para el caso de **n=28**, nuestro algoritmo solo necesitó **una cámara adicional** frente al óptimo (18 frente a 17). Esto representa un error de solo el **5.8%**, lo cual es un resultado excelente para un algoritmo que se ejecuta de forma casi instantánea.
- El "error relativo" tiende a estabilizarse o incluso disminuir a medida que el problema crece. Esto sugiere que para complejos de vigilancia muy grandes, nuestra heurística es extremadamente fiable.

2. Compromiso Eficacia-Eficiencia

La verdadera ventaja de nuestro algoritmo no es superar al óptimo en número de cámaras, sino en el **coste computacional** para llegar a esa cifra:

- **El Algoritmo Óptimo (R1):** Aunque devuelve el número mínimo de cámaras, su coste en tiempo es inasumible. Para $n=28$, el proceso tarda **37.46 segundos**. Siguiendo una progresión exponencial, para $n=40$ podría tardar días en dar una respuesta.
- **Vuestro Algoritmo Voraz (R2):** La solución de 18 cámaras se obtiene en milisegundos. En un entorno de ingeniería real, es preferible instalar una cámara extra (un coste fijo pequeño) que esperar días a que un ordenador calcule la posición exacta (un coste operativo inmenso).

3. Estabilidad de la Heurística

La comparativa muestra que el algoritmo voraz es "estable". No hay ningún salto brusco donde el algoritmo dé un resultado disparatado (como 50 cámaras donde el óptimo pide 10). Esto demuestra que la **Función de Selección** (elegir el nodo que cubre más pasillos) es una estrategia muy robusta para el problema del *Vertex Cover*.

Análisis de la Eficiencia (Escalabilidad)

Aquí es donde los datos muestran la verdadera potencia del método voraz frente a la fuerza bruta:

- **Crecimiento Exponencial del Óptimo:** El tiempo de la fuerza bruta en resultados2. Para $n=24$ tarda casi 2 segundos , pero para $n=28$ (solo 4 nodos más) el tiempo salta a **37.46 segundos**. Esto confirma que su complejidad es de orden exponencial, probablemente $O(2^n)$, lo que lo hace inútil para grafos grandes.
- **Velocidad del Voraz:** Nuestro algoritmo para $n=28$ tarda un tiempo despreciable (menos de 0.01s), ya que su complejidad es polinomial $O(n_v^2)$ según vuestro análisis).
- **Punto de ruptura:** Según los datos de resultados2, el algoritmo óptimo deja de ser práctico a partir de $n=30$, donde el tiempo de espera superaría los minutos o incluso horas. Nuestro voraz, en cambio, podría manejar grafos de $n=1000$ en menos de un segundo.